



Observability for Debugging and Improvement

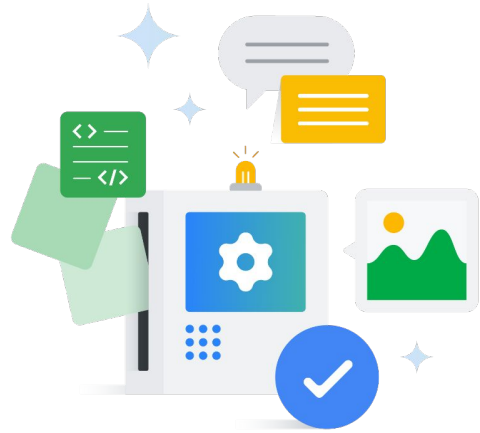
Welcome to Module 3: Observability for Debugging and Improvement.

In this module, we focus on observability—the discipline of understanding how an agent behaves in production.

AI agents are complex systems involving models, tools, prompts, and external services. Without proper observability, these systems become extremely difficult to debug and optimize. We'll look at how to instrument agents so we can measure, debug, and continuously improve them.

In this module, you learn to ...

- 01 Identify challenges addressed by observability
- 02 Instrument an ADK agent with structured logs
- 03 Enable OpenTelemetry tracing on Agent Runtime and Cloud Run
- 04 Leverage BigQuery and Data Studio for visualization

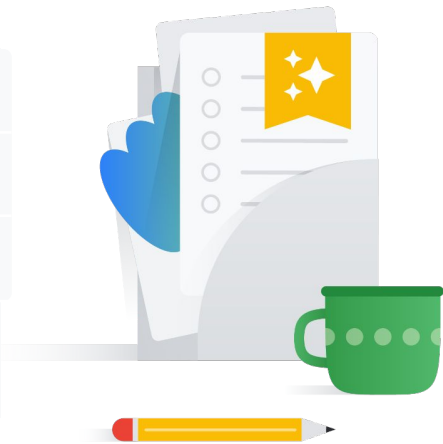


In this module, we'll focus on four main capabilities.

1. First, we'll understand the challenges observability solves in AI agent systems.
2. Second, we'll instrument an ADK agent with structured logging so we can inspect agent behavior.
3. Third, we'll enable OpenTelemetry tracing to track the end-to-end execution path of an agent request.
4. Finally, we'll use BigQuery and Data Studio to analyze observability data and visualize agent performance.

Topics

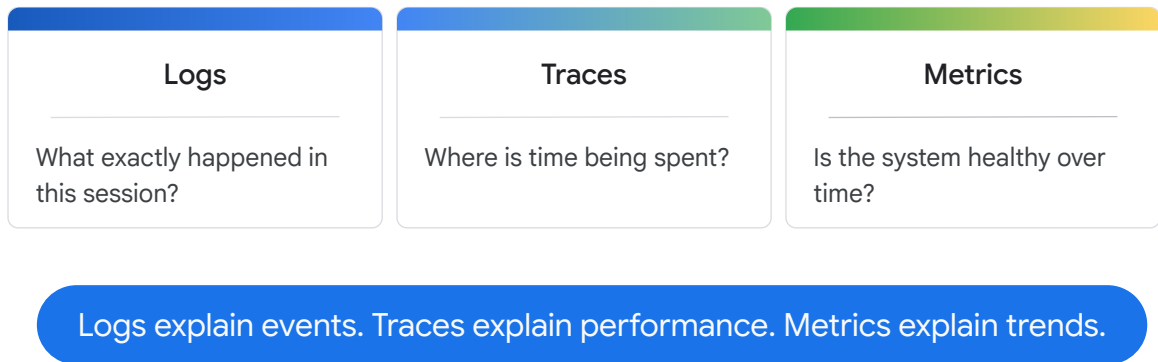
- | | |
|----|--|
| 01 | Observability review |
| 02 | Logging with agent callback logging |
| 03 | Logging and tracing with OpenTelemetry |
| 04 | Lab: Instrument and Debug Agents with Cloud Logging, and Cloud Trace |



The module is organized into four sections.

1. We begin with an observability overview.
2. Then we explore callback logging for agents.
3. Next we examine OpenTelemetry tracing for performance analysis.
4. Finally, we'll complete a hands-on lab where you instrument and debug an agent using Cloud Logging and Cloud Trace.

Observability for AI Agents: The three signals



Observability is typically built on three core signals: Logs, metrics, and traces.

- Logs provide detailed event records describing what happened during execution.
- Metrics track aggregated trends over time such as latency, error rates, or cost.
- Traces show how time is spent across components of a system.

Together these signals allow us to understand both individual agent sessions and overall system behavior.

Common business challenges addressed by observability



Observability turns black-box agents into measurable systems

Google Cloud

- Why did my agent give this specific answer?
- Which tool call is making my agent slow?
- How many users are experiencing errors?
- Is the cost per agent session increasing?
- How can I correlate user feedback to a specific agent interaction?

Proprietary & Confidential

5

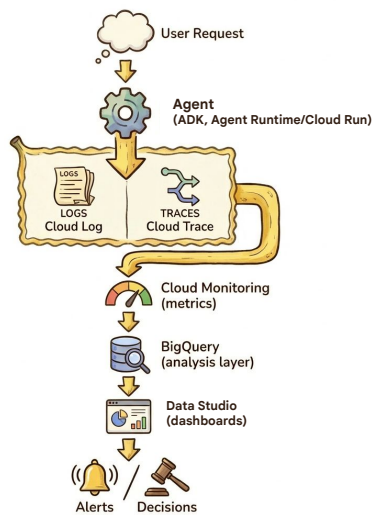
Observability is not just a technical concern—it solves real business questions.

For example:

- Why did the agent produce a specific answer?
- Which tool call is making the agent slow?
- How many users are encountering errors?
- Is the cost per session increasing?
- And how can we connect user feedback to a specific agent interaction?

Observability helps transform AI agents from black-box systems into measurable systems.

Observability architecture for AI Agents



Observability is not a tool —
it is a pipeline from execution
to decision

Observability should be thought of as a pipeline rather than a single tool.

Data flows from the agent's execution environment through several layers.

- First, the agent emits logs and traces.
- Those signals are captured by systems like Cloud Logging and Cloud Trace.
- Metrics are aggregated in Cloud Monitoring.
- Then the data can be exported into BigQuery for deeper analysis and visualized in dashboards such as Data Studio.

This pipeline allows teams to move from raw telemetry to operational decisions.

Logging: Cloud Logging

- Fully managed, scalable logging service
- Logs are automatically ingested when agents run on Google Cloud environments such as Cloud Run or Agent Runtime
- Supports filtering by `trace_id` and `session_id`
- Supports powerful querying and analysis
 - Log explorer
 - Log analytics
 - Sink logs to BigQuery or third-party analysis tools
- Enables reconstruction of session-level execution from logs

Cloud Logging is Google Cloud's managed logging service.

It automatically ingests logs from services such as Cloud Run and Agent Runtime.

Logs can be filtered using identifiers like `trace_id` or `session_id`, which makes it easier to reconstruct a specific user interaction.

Logs can also be exported to systems like BigQuery for deeper analysis or connected to third-party observability platforms.

Logging: Logging configuration in ADK

- Agent Development Kit (ADK) can be configured for structured logging
- Emits structured lifecycle events:
 - session_start
 - model_call
 - tool_call
 - final_response
- Logs include important context like trace IDs
- Structured logs are automatically parsed when emitted in supported formats (e.g., JSON or via logging libraries)

The Agent Development Kit supports structured logging out of the box.

Agents emit lifecycle events such as:

- Session start
- Model call
- Tool call
- Final response

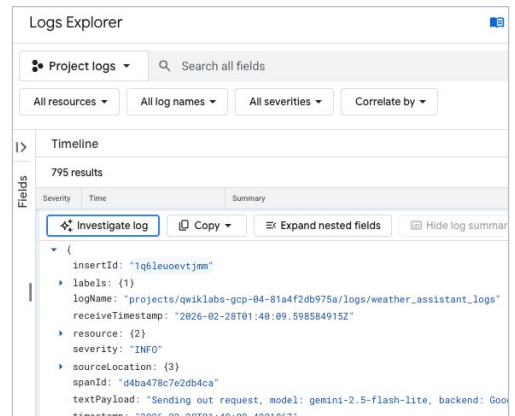
These logs include important metadata such as trace IDs, which allow events to be correlated across services.

When running on Google Cloud, structured logs are automatically formatted as JSON and parsed by Cloud Logging.

Logging example: Debugging with logs

Support ticket: "The agent failed for my order"

- Filter logs in Log Explorer to locate users session
- Review the sequence of structured logs for that session
- Identify an error log from a tool call with malformed input
- The log entry contains the input that caused the failure



Let's look at a practical example.

Imagine a support ticket where a user says the agent failed to process their request.

Using Cloud Logging, we can filter logs by the user's session ID.

This allows us to replay the sequence of events for that session.

By inspecting the logs, we might discover that a tool call failed because of malformed input.

The log entry can reveal exactly what input caused the failure, allowing developers to quickly fix the issue.

Tracing: OpenTelemetry (OTel)

- An open-source, vendor-neutral observability framework
- Provides APIs and SDKs for instrumenting applications
- A 'trace' represents the end-to-end journey of a request
- A trace is composed of 'spans' (e.g., API call, tool execution)

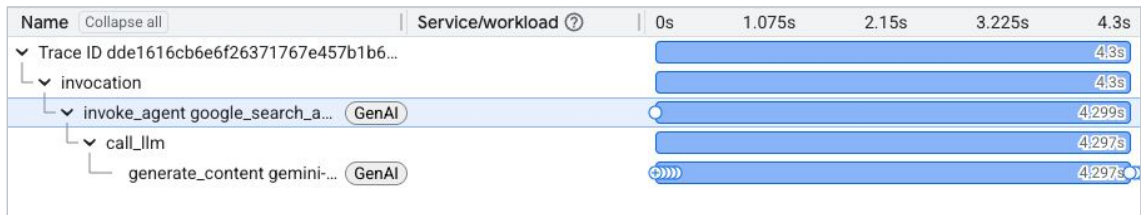
Logs help explain what happened, but they don't always show where time is being spent.

That's where tracing comes in.

- OpenTelemetry is an open-source framework used to instrument applications and collect observability data.
- A trace represents the entire lifecycle of a request.
- Each step in the process is represented by a span, such as a model call or tool invocation.

Tracing: Cloud Trace

- Google Cloud's distributed tracing system
- Ingests trace data from OpenTelemetry and other sources
- Visualizes traces as waterfall diagrams
- Helps identify performance bottlenecks



Cloud Trace is Google Cloud's distributed tracing system.

It visualizes traces as waterfall diagrams that show how time is spent across different components.

This makes it easier to identify bottlenecks in agent execution.

For example, we can see whether latency comes from the LLM, a tool call, or a database query.

Tracing example: Understanding performance with traces

- A user complains an agent is slow
- Find the trace for the user's request in Cloud Trace
- The waterfall diagram shows a span for an invocation
- Drill down into the span to see it's making a slow tool call, DB query, etc.

Traces shift the optimization conversation from “the agent is slow” to “this tool call is slow”

Suppose users report that an agent feels slow.

Instead of guessing where the delay occurs, we can inspect the request trace.

The trace shows the full chain of execution, including each tool call.

If a particular span shows significantly longer duration, we've found the source of the bottleneck.

Tracing allows us to move from vague complaints like “the agent is slow” to precise insights such as “this database query is slow.”

Monitoring

- Monitoring moves from debugging individual sessions to managing system health
 - Monitoring answers: “Is production healthy right now?”
- Cloud Monitoring collects and visualizes time-series metrics
- Infrastructure metrics
 - CPU
 - Memory
 - Request latency
- Custom agent metrics must be emitted or derived from telemetry pipelines:
 - Token usage
 - Tool frequency
 - User feedback score (e.g., thumbs up/down)
 - Cost per session
- Set up alerts to be notified of anomalies



Monitoring shifts the focus from individual debugging sessions to system-wide health.

Cloud Monitoring tracks time-series metrics such as CPU usage, memory utilization, and request latency.

For agents, we may also track custom metrics such as token usage, tool frequency, feedback scores, or cost per session.

Alerts can be configured to notify teams when anomalies occur.

Monitoring answers “Is the system healthy?”

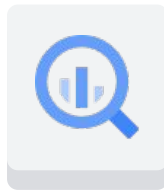
Logging answers “What happened?”

Tracing answers “Where is it slow?”

Consuming data: BigQuery

- BigQuery is a serverless data warehouse for large-scale analytics
- Export logs, traces, and evaluation results to BigQuery
- Enables complex SQL queries across all observability data
 - Join logs with evaluation results to find common failure modes

Example analysis: Identify the top three prompts correlated with negative feedback



BigQuery allows teams to perform large-scale analytics across observability data.

Logs, traces, and evaluation results can all be exported into BigQuery.

This allows complex queries across multiple datasets.

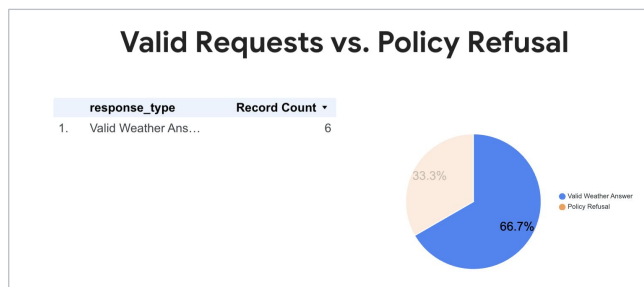
For example, we might analyze which prompts correlate with negative user feedback or identify patterns in tool failures.

There is also the BigQuery Agent Analytics plugin for ADK:

<https://google.github.io/adk-docs/integrations/bigquery-agent-analytics/>

Consuming data: Data Studio

- Free data visualization and dashboarding tool
- Bridges technical observability and business reporting
- Connects directly to BigQuery and other data sources
- Build interactive dashboards to track agent KPIs
- Share insights with business stakeholders



Google Cloud

Proprietary & Confidential

15

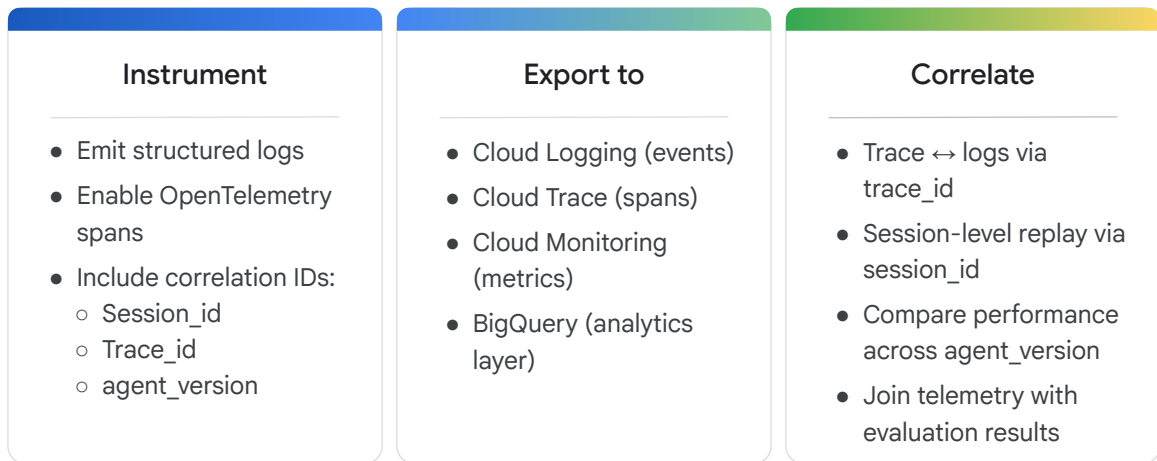
Data Studio provides a visualization layer for observability data.

It connects directly to BigQuery and other sources.

Dashboards can display key agent metrics such as latency, success rate, token usage, or cost.

These dashboards are particularly useful for sharing insights with product teams and business stakeholders.

Observability architecture pattern for agents



Google Cloud

Proprietary & Confidential 16

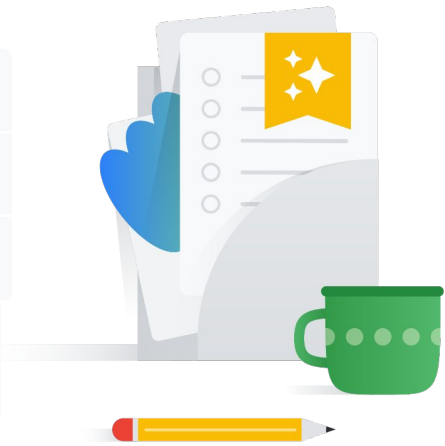
A common observability architecture follows three phases.

- First, instrumentation, where logs and traces are emitted by the application.
- Second, export, where telemetry is captured by services like Cloud Logging and Cloud Trace.
- Third, correlation and analysis, where telemetry data is joined across systems using identifiers like session IDs and trace IDs.

This approach enables teams to reconstruct agent behavior and compare performance across agent versions.

Topics

- | | |
|----|--|
| 01 | Observability review |
| 02 | Logging with agent callback logging |
| 03 | Logging and tracing with OpenTelemetry |
| 04 | Lab: Instrument and Debug Agents with Cloud Logging, and Cloud Trace |



Agent callback logging

- Callbacks are “hooks” that trigger at specific points in an agent’s lifecycle
 - `before_model_callback`, `after_model_callback`, `before_tool_callback`, etc.
 - When used for logging, you are manually capturing data as it happens
- Pros
 - Best for rapid prototyping
 - Conditional Logic: only log when a specific error occurs or a specific tool is used
 - Short-circuiting: Unlike OpenTelemetry, callbacks can actually change the agent’s behavior
 - I.e., stopping a run if a guardrail is triggered
- Cons
 - It is “siloed”
 - If you have multiple agents, it’s hard to see the big picture
 - Adds overhead to the agent’s main execution thread

Another technique for observability is callback logging.

Callbacks are hooks that trigger at specific points during an agent’s lifecycle.

For example, before a model call or after a tool invocation.

Callbacks allow developers to capture custom logging information or even alter agent behavior.

This approach is particularly useful for debugging or implementing guardrails.

Callback logging: Create the logging functions (Python)

callback_logging.py

```
import logging
from google.adk.agents.callback_context import CallbackContext
from google.adk.models import LlmResponse, LlmRequest

def log_query_to_model(callback_context: CallbackContext, llm_request: LlmRequest):
    if llm_request.contents and llm_request.contents[-1].role == 'user':
        for part in llm_request.contents[-1].parts:
            if part.text:
                logging.info("[query to %s]: %s", callback_context.agent_name, part.text)

def log_model_response(callback_context: CallbackContext, llm_response: LlmResponse):
    if llm_response.content and llm_response.content.parts:
        for part in llm_response.content.parts:
            if part.text:
                logging.info("[response from %s]: %s", callback_context.agent_name, part.text)
            elif part.function_call:
                logging.info("[function call from %s]: %s", callback_context.agent_name, part.function_call.name)
```

This slide shows an example Python implementation of callback logging.

The functions capture the user's query before the model call and log the model's response afterward.

These logs can then be sent to Cloud Logging for inspection.

This approach allows developers to capture highly specific information about agent behavior.

Callback logging: Instrument the agent (Python)

agent.py

```
# Example: Log user prompts and model responses to Cloud Logging
...
import logging
from .callback_logging import log_query_to_model, log_model_response

root_agent = Agent(
    name="google_search_agent",
    description="Answer questions using Google Search.",
    model="gemini-2.5-flash-lite",
    instruction="You are an expert researcher. You stick to the facts.",

    before_model_callback=log_query_to_model,
    after_model_callback=log_model_response,

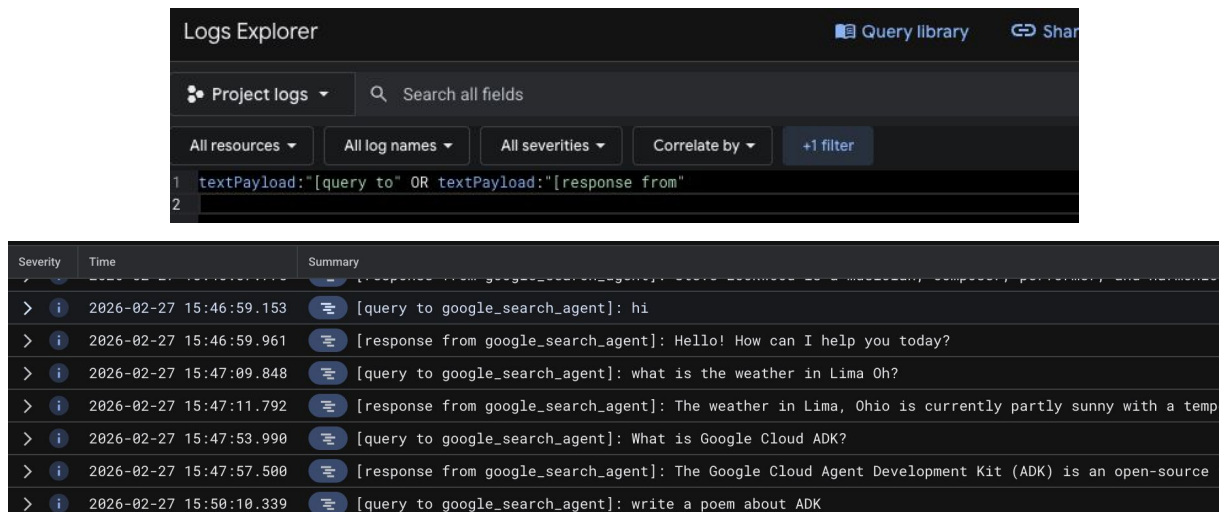
    tools=[google_search]
)
```

To enable callback logging, we attach logging functions to the agent configuration.

In this example, the `before_model_callback` captures the user prompt, and the `after_model_callback` logs the model response.

This allows us to observe the agent's interaction with the model in real time.

Logs routed to Cloud Logging



The screenshot displays the Google Cloud Logs Explorer interface. At the top, the 'Logs Explorer' header is visible, along with a 'Query library' link and a 'Share' icon. Below the header, there's a search bar with the placeholder text 'Search all fields'. To the left of the search bar is a dropdown menu labeled 'Project logs'. Below the search bar, there are several filter buttons: 'All resources', 'All log names', 'All severities', 'Correlate by', and a '+1 filter' button. The main content area shows a list of logs. The first two lines of the log list are highlighted, showing a search filter: `1 textPayload:["query to" OR textPayload:["response from"` and `2`. Below this, a table of logs is displayed with columns for 'Severity', 'Time', and 'Summary'. The table contains several log entries, each with an information icon, a timestamp, and a summary of the log message.

Severity	Time	Summary
> i	2026-02-27 15:46:59.153	[query to google_search_agent]: hi
> i	2026-02-27 15:46:59.961	[response from google_search_agent]: Hello! How can I help you today?
> i	2026-02-27 15:47:09.848	[query to google_search_agent]: what is the weather in Lima Oh?
> i	2026-02-27 15:47:11.792	[response from google_search_agent]: The weather in Lima, Ohio is currently partly sunny with a temp
> i	2026-02-27 15:47:53.990	[query to google_search_agent]: What is Google Cloud ADK?
> i	2026-02-27 15:47:57.500	[response from google_search_agent]: The Google Cloud Agent Development Kit (ADK) is an open-source
> i	2026-02-27 15:50:10.339	[query to google_search_agent]: write a poem about ADK

Once the agent emits logs, they appear in Cloud Logging.

Developers can search logs using the Log Explorer interface.

This shows a search filter in cloud logging that looks for logs that start with “query to” or “response from”.

Structured logging makes it easy to filter by agent name, session ID, or other metadata.

This enables quick investigation of specific agent sessions.



Logging a model's prompt and response is great for debugging, but could cause **Sensitive Personal Information (PII) to leak into your logs** or trace backends

While logging prompts and responses is useful for debugging, it also introduces privacy concerns.

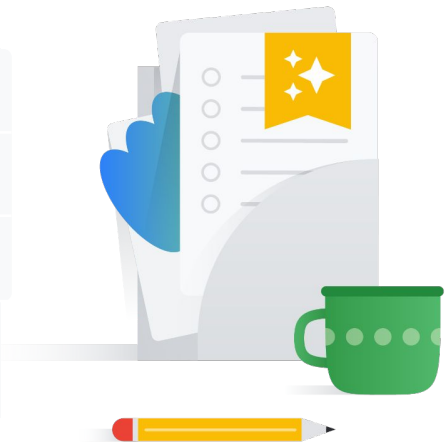
Sensitive personal information could accidentally appear in logs.

Organizations should apply policies to redact or sanitize sensitive data before logging it.

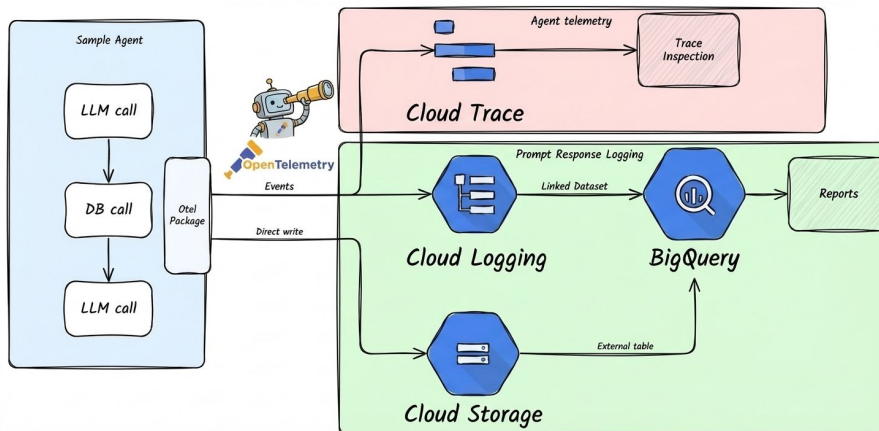
This ensures observability does not introduce security or compliance risks.

Topics

- | | |
|----|--|
| 01 | Observability review |
| 02 | Logging with agent callback logging |
| 03 | Logging and tracing with OpenTelemetry |
| 04 | Lab: Instrument and Debug Agents with Cloud Logging, and Cloud Trace |



Agent execution telemetry



Logs and traces can be correlated via `trace_id` when trace context is properly propagated

In a fully instrumented system, logs and traces are automatically correlated using identifiers like `trace_id`.

This allows developers to connect events across services and reconstruct the entire execution path of a request.

This correlation is critical for understanding complex agent workflows.

Controlling message content logging

- OpenTelemetry instrumentation for LLMs captures metadata like latency, model name, and token counts
 - The conversation text is redacted to prevent sensitive information from leaking into logs or trace backends
- Setting the environment variable:

```
OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT = True
```

- Captures:
 - The user prompt
 - The system instructions
 - The model response
 - Tool/function arguments

OpenTelemetry instrumentation captures metadata such as latency, token counts, and model names.

By default, message content may be redacted to prevent sensitive information from appearing in logs.

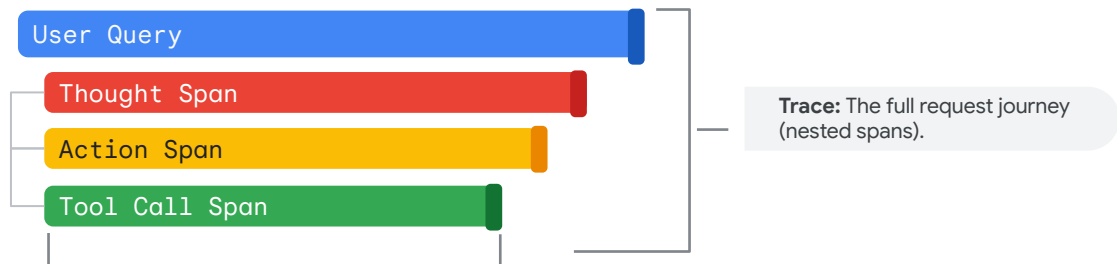
Developers can enable message capture by setting a configuration flag, but this should only be done with proper safeguards.

This can be controlled by setting the value of the Environment variable: `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT`. Setting to true, instructs the instrumentation to stop redacting the conversation and instead capture the full text of:

- The User Prompt: Exactly what the user asked
- The System Instructions: The hidden "persona" or developer-defined rules
- The Model Response: The exact text the AI generated
- Tool/Function Arguments: The JSON data the AI sent to your tools

Tracing spans

- The fundamental unit of measurement in tracing is the **span**
 - It's a named block of execution time
- The steps in the agent's logic—decisions, tool calls, etc.—can be captured as a span
- A full trace is simply the collection of all these nested spans for one user request



The fundamental unit in tracing is the span.

A span represents a block of execution time.

For example, a model call, a tool invocation, or a reasoning step.

A complete trace consists of multiple nested spans that together represent the full request journey.



Cloud Trace



Collects latency data from distributed applications and displays it in the Google Cloud console.



Supports OpenTelemetry method for instrumentation

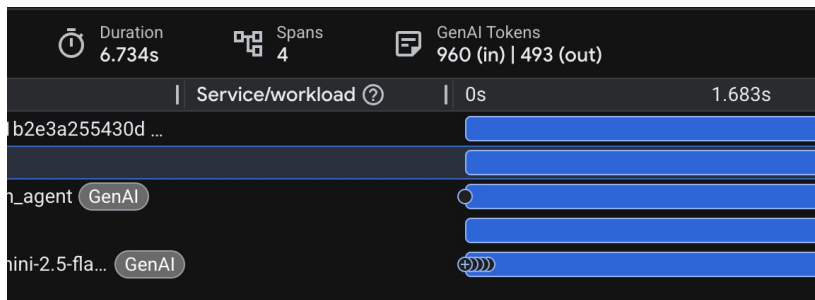
Cloud Trace, based on the tools Google uses on its production services, is a tracing system that collects latency data from your distributed applications and displays it in the Google Cloud console.

Trace can capture traces from applications deployed on App Engine, Compute Engine VMs, and Google Kubernetes Engine containers and ADK.

[Agent Development Kit](#) - Agent Observability with Cloud Trace.

Tracing: OpenTelemetry configuration with ADK

- ADK has built-in OpenTelemetry support
- Automatically creates spans for instrumented operations such as model and tool calls
- When deployed on Cloud Run or Agent Runtime, traces can be sent to Cloud Trace with little effort
 - Also displays GenAI token count (in and out)



The Agent Development Kit includes built-in OpenTelemetry support.

It automatically generates spans for model calls and tool executions.

When deployed on Cloud Run or Agent Runtime, these traces can be exported directly to Cloud Trace with minimal configuration.

This gives teams visibility into token usage and execution latency.

ADK Observability on Agent Runtime vs Cloud Run

Agent Runtime

- Built-in OpenTelemetry instrumentation
- Automatic span creation for:
 - Model calls
 - Tool calls
 - Agent lifecycle events
- Automatic export when deployed with ADK:
 - Cloud Trace
 - Cloud Logging
- No collector configuration required
- No infrastructure management

Cloud Run

- OpenTelemetry enabled via ADK flag:
 - `--otel_to_cloud`
- Automatic export when deployed with ADK:
 - Cloud Trace
 - Cloud Logging
- Supports custom spans
- Access to infrastructure metrics:
 - CPU
 - Memory
 - Concurrency
 - Autoscaling

Agent Runtime and Cloud Run handle observability slightly differently.

Agent Runtime includes built-in instrumentation and automatically exports telemetry. Automatic export when deployed with ADK on supported Google Cloud environments

Cloud Run also supports observability but requires enabling telemetry flags during deployment.

The flags are provided by the ADK deployment tooling, not native Cloud Run.”

Cloud Run additionally provides infrastructure-level metrics such as CPU and concurrency.

Enabling OpenTelemetry for agents on Cloud Run

- For full OTEL (Traces, Logs, GenAI data)
 - Deploying with CLI, specify ADK flag: `--otel_to_cloud` :
 - **`adk deploy cloud_run ... --otel_to_cloud`**
 - If deploying the app programmatically (i.e. `get_fast_api_app`) set `otel_to_cloud=True`

```
app = get_fast_api_app(  
    agents_dir=".",  
    otel_to_cloud=True,  
)
```

- No code required for default spans
 - Custom instrumentation requires additional code

OpenTelemetry can be enabled during deployment.

For Cloud Run deployments, the `otel_to_cloud` flag automatically exports traces and logs.

No additional instrumentation code is required for basic spans.

This simplifies observability setup.

Example: Custom span in Cloud Run (Python)

- Custom spans help instrument internal logic beyond default ADK spans

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

def lookup_inventory(item_id):
    with tracer.start_as_current_span("lookup_inventory"):
        result = database_query(item_id)
    return result
```

Developers can also create custom spans to instrument internal logic.

For example, wrapping a database query in a span allows developers to measure its latency.

Custom spans help capture performance insights that automatic instrumentation might miss.

Callback Logging vs OpenTelemetry

Feature	Callback Logging	OpenTelemetry
Goal	Local debugging/control flow	Full-stack observability
Data scope	Only what you code	Automatic (LLM calls, tools, spans)
Impact	Runs in request thread	Non-blocking export pipeline (asynchronous)
Complexity	Low (manual instrumentation)	Moderate
Use case	Guardrails, custom control	Production observability

Callback logging and OpenTelemetry serve different purposes.

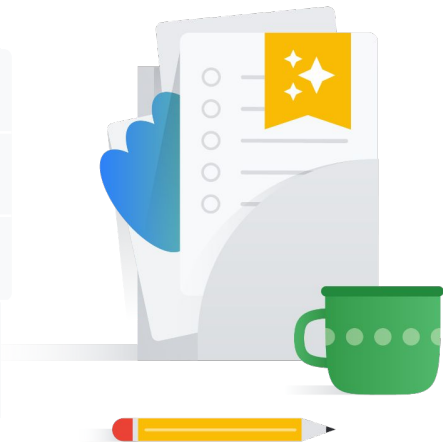
Callback logging is ideal for targeted debugging and implementing guardrails.

OpenTelemetry provides system-wide observability with asynchronous telemetry export.

In production systems, both approaches are often used together.

Topics

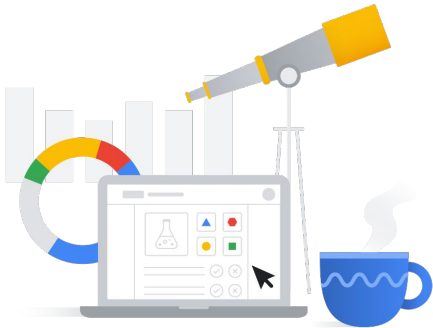
- | | |
|----|--|
| 01 | Observability review |
| 02 | Logging with agent callback logging |
| 03 | Logging and tracing with OpenTelemetry |
| 04 | Lab: Instrument and Debug Agents with Cloud Logging, and Cloud Trace |



Next we'll move into the hands-on lab.

In this exercise, you will instrument an agent with logging and tracing to observe how it behaves during execution.

Lab: Instrument and Debug Agents with Cloud Logging and Cloud Trace



01 Add callback logging to an agent

02 Analyze logs in Big Query and visualize in Data Studio

03 Enable Open Telemetry for the agent

04 View agent traces in Google Cloud Trace

In this lab, you:

- Add callback logging to an agent.
- Analyze logs using BigQuery and visualize results in Data Studio.
- Enable OpenTelemetry tracing.
- And finally inspect traces using Cloud Trace.

This will demonstrate how observability tools help debug and optimize agent systems.

Questions and answers



If anything about logging, tracing, or monitoring is unclear, now is a good time to discuss it.



Thank you.
Module 3 is complete.